Architecture Modeling Track

Lab T-AM2: Cycle-Approximate Timing Model

Timing in the Model, Calibration and Cycle Correlation

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Extend the T-MOD cache model with a timing layer that predicts the cycles and stall cycles of every access, calibrate its parameters from RTL measurements, correlate predicted and measured cycle counts within a stated error, and use the calibrated model for what-if and sensitivity studies that the RTL cannot answer quickly.

Duration	2 weeks (approximately 35–40 hours)
Prerequisites	Lab T-AM1 (traces, `tb_explore`, `tb_replay`, exact count correlation); Lab T-MOD; Labs R4 and V3 (controller FSM, performance counters, latency statistics)
Designs	`cache_timing_model` and `timing_cfg` (new, in `cache_model_pkg`); calibration, correlation and what-if scripts in `dv/model/explore/` (new); `cache_model` and `dcache` used unchanged
Reused in	Final lab of the architecture-modeling track; the method (calibrate, correlate, explain residuals) is the one used for core-level performance models

Contents

1 Overview **3**

2 From Counts to Cycles **5**

2.1 The Spectrum of Performance Models 5

3 Timing of the Blocking Cache **5**

3.1 Anatomy of a Miss 5

3.2 Timing Parameters 5

3.3 Cycle Formulas 6

4 The Timing Model **7**

4.1 Composition, Not Modification 7

4.2 API and Implementation 7

4.3 Requirements 8

5 Calibration **9**
5.1 Micro-Benchmarks . . . 9
5.2 Fitting . . . 9

6 Cycle Correlation **10**
6.1 Error Metrics . . . 10
6.2 Residual Analysis Access by Access . . . 11

7 What-If Studies **12**
7.1 Memory Latency . . . 12
7.2 A Non-Blocking Cache with MSHRs . . . 12

8 Sensitivity Analysis **13**

9 Limits of Cycle-Approximate Models **14**

10 Industrial Reference: the CVA6 Performance Model **14**

11 Requirements and Tasks **15**

12 Repository and Flow **15**

13 Deliverables **16**

14 Assessment **17**

15 Review Questions **17**

16 References **18**

1 Overview

Lab T-AM1 answered *how often* the cache misses. Architects need to know *how long* a workload takes: a configuration with a slightly lower hit rate can be faster if its misses are cheaper, and a change in memory latency can reverse a ranking that looked clear in hit-rate terms. Hit rates are functional and the T-MOD model predicts them exactly; cycles depend on the micro-architecture, and predicting them needs a model of time.

There is a spectrum of answers. The RTL is exact but slow, and exists only for the configurations someone has designed. A **cycle-accurate** model reproduces every cycle of the RTL — it is essentially a second implementation, as expensive to write and to keep in sync as the RTL. A **cycle-approximate** model adds a small number of timing parameters (latencies, occupancies) to a transaction-level model. It is nearly as fast as the functional model, it covers configurations and memory systems that have no RTL, and — when calibrated and correlated — it is accurate to a few percent. Most performance models used for architecture decisions are of this kind.

In this lab you build such a model for the R4 cache, in four steps:

1. **Model**: a timing layer around the T-MOD model, which stays unchanged; the layer turns each predicted result into stall cycles using latency parameters.
2. **Calibrate**: measure the parameters on the RTL with micro-benchmark traces, and explain each one by the R4 controller FSM.
3. **Correlate**: compare predicted and measured total cycles and stall cycles on the T-AM1 workload traces, per trace, with an error target, and explain every residual.
4. **Use**: what-if studies (memory latency, a non-blocking cache with miss status holding registers), sensitivity analysis, and an honest statement of the model's limits.

Figure 1 shows how the pieces connect. The essential rule is the separation of the data: the micro-benchmark traces are used *only* to fit the parameters, the T-AM1 workload traces *only* to validate them.

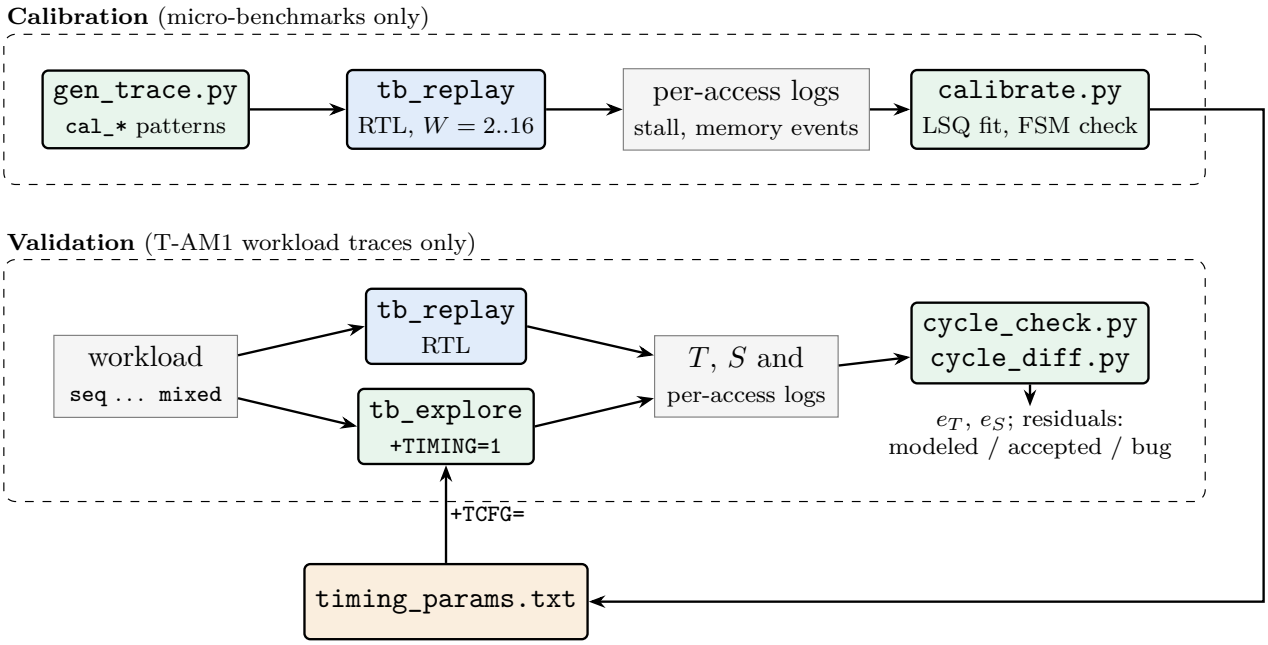


Figure 1: Calibration and correlation flow. Calibration (top) fits the parameters to RTL micro-benchmarks; validation (bottom) runs the RTL and the timing model on *different* traces and compares total cycles, stall cycles and the per-access timelines.

TEST TERM | Model correlation test (cycles)

> The same workload trace on the RTL and on the calibrated timing model; total cycles and stall cycles are compared per trace against an error target ($|e_T| < 2\%$ here), and the per-access timelines are aligned to find where they differ. A pass proves the timing model for the correlated range (configurations, memory latency, workloads) — not beyond it, which is why every what-if result is labeled.

The industrial reference for this flow is the performance model of the CVA6 core (Section 10): a Python model of the core's timing and a script that compares its cycle numbers with RTL traces, instruction by instruction — the same idea, at the level of a whole core.

Where this lab sits. It builds on T-AM1 (traces, the replay testbench and its per-access log, the exact count correlation) and on the T-MOD model, which is reused by composition and not modified. It reads the R4 RTL and its counters, and uses the V3 latency statistics as a cross-check. It is the last lab of the architecture-modeling track.

NOTE | Naming.

> In the team, "performance model" means the C++ models of the whole processor (Sniper and similar simulators), owned by the performance team. This lab builds a SystemVerilog cycle-approximate timing model of one block, the cache, to learn the methods those models depend on: calibration, correlation against RTL and what-if studies.

By the end of this lab you should be able to:

1. Derive the cycle cost of hits, clean misses, dirty misses and flushes of a blocking cache from its controller and memory protocol, as formulas in a few parameters.
2. Add timing to a transaction-level model by composition, without changing the functional model, and explain why.
3. Calibrate model parameters from RTL measurements with directed micro-benchmarks, separating calibration data from validation data.
4. Correlate cycle predictions with the RTL, report the error per trace, and find the source of residuals access by access.
5. Use the model for what-if studies (memory latency, non-blocking misses) and sensitivity analysis, and state the confidence of each result.
6. Explain the limits of cycle-approximate modeling compared with cycle-accurate models and RTL.

2 From Counts to Cycles

2.1 The Spectrum of Performance Models

Model	Speed (accesses/s)	Cycle error	Typical use
Analytical formula (AMAT)	instant	5–30%	First estimates, sanity checks
Trace-driven functional (T-AM1)	10^6–10^7	counts exact, no cycles	Hit rates, traffic, three-C
Cycle-approximate (this lab)	10^6–10^7	1–10%	Architecture trade-offs, what-if
Cycle-accurate model	10^4–10^5	0% (by construction)	Micro-architecture tuning; golden timing
RTL simulation	10^3–10^5	reference	Implementation, sign-off
FPGA emulation	10^6–10^8	reference	Software on real RTL, long workloads

The numbers are orders of magnitude, not measurements; you measure your own in T-AM1 and in this lab. The point of the table is that every row trades accuracy for speed and coverage, and a performance team uses several rows at once: the fast model for breadth, the RTL for correlation.

3 Timing of the Blocking Cache

3.1 Anatomy of a Miss

The R4 cache accepts a request only when it hits; a missing request is held by the master (the `mem_if` protocol of Lab R3) while the controller evicts and refills, and is then accepted as a hit. The memory side is `arb_mem`: a write is accepted and completed at one edge, a read is accepted at one edge and its response arrives exactly one cycle later, and with the loader idle there is no arbitration delay. The R4 counter `stall_cycles` counts the cycles with `cpu.req_valid && !cpu.req_ready`; this lab predicts it, and the total cycles of a replay (first request to `flush_done`), the two quantities T-AM1 did not correlate. Figure 2 decomposes a dirty miss with `line_words = 4` into the parameters of the timing model.

3.2 Timing Parameters

Parameter	Symbol	Meaning (cycles)
`c_det`	c_{det}	From the first cycle a missing request is presented to the first memory request
`t_wb`	t_{wb}	Interval between write-back requests (1 if one write per cycle)
`c_e2r`	c_{e2r}	Turnaround from the last write-back request to the first refill request
`t_rf`	t_{rf}	Interval between refill read requests (1 if pipelined; $1 + L_{\text{mem}}$ if the controller waits for each response)
`l_mem`	L_{mem}	Memory read latency, request acceptance to response ($= 1$ for `arb_mem`)
`c_ret`	c_{ret}	From the last refill response to the acceptance of the held request
`c_fl`	c_{fl}	Flush: cycles per line scanned (clean or invalid)
`c_fd`	c_{fd}	Flush: extra cycles per dirty line in addition to Wt_{wb}

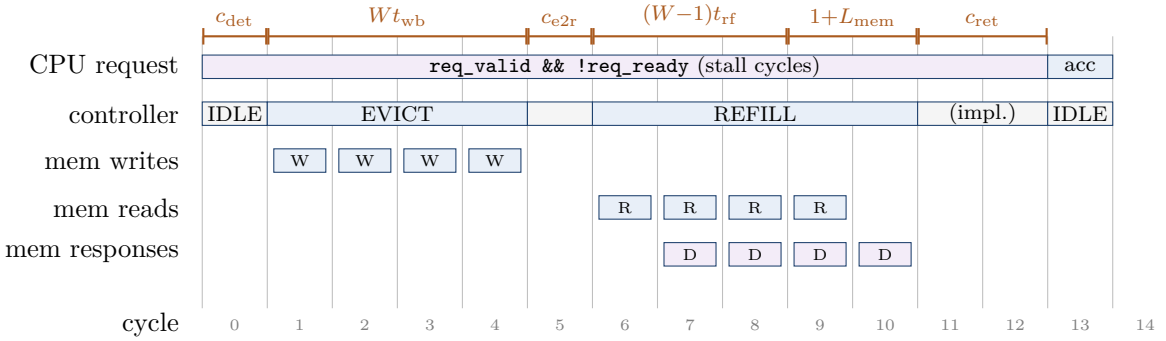


Figure 2: A dirty miss with $W = 4$, decomposed into timing parameters. The cycle numbers are illustrative: the length of each segment of *your* R4 controller is what calibration (Section 5) measures. The grey "impl." segment stands for whatever your controller does between the last refill response and acceptance (for example a WAIT state, or writing the last word into the data array).

3.3 Cycle Formulas

With $W =$ `line_words`, the stall cycles of one access (cycles during which the request is presented but not accepted) are

$$
\begin{aligned}
s_{\text{hit}} &= 0, \\
S_c(W) &= c_{\text{det}} + (W-1)\,t_{\text{rf}} + 1 + L_{\text{mem}} + c_{\text{ret}} \;=\; a_c + b_c W, \qquad (1)\\
S_d(W) &= S_c(W) + W\,t_{\text{wb}} + c_{\text{e2r}} \;=\; a_d + b_d W. \qquad (2)
\end{aligned}
$$

Both penalties are *linear* in W; this is what makes calibration from a few line sizes possible (Section 5). For a trace of N accesses issued back to back, with n_c clean and n_d dirty misses, and a final flush of a cache with N_L lines of which n_{fd} are dirty:

$$
\begin{aligned}
\mathit{stall_cycles} &= n_c\,S_c(W) + n_d\,S_d(W), \qquad (3)\\
T_{\text{flush}} &= N_L\,c_{\text{fl}} + n_{\text{fd}}\,(W\,t_{\text{wb}} + c_{\text{fd}}), \\
T &= N + \mathit{stall_cycles} + T_{\text{flush}} + c_0, \qquad (4)
\end{aligned}
$$

where c_0 is a constant (start and end of the replay, the pulse to `flush_done`). The average access time is $AMAT = 1 + \mathit{stall_cycles}/N = 1 + m_c S_c + m_d S_d$, the formula of T-AM1, now with predicted instead of measured penalties (Figure 3 shows which terms add up for each kind of access). Note that n_c, n_d and n_{fd} come from the functional model and are exact (T-AM1); all timing uncertainty is in the parameters.

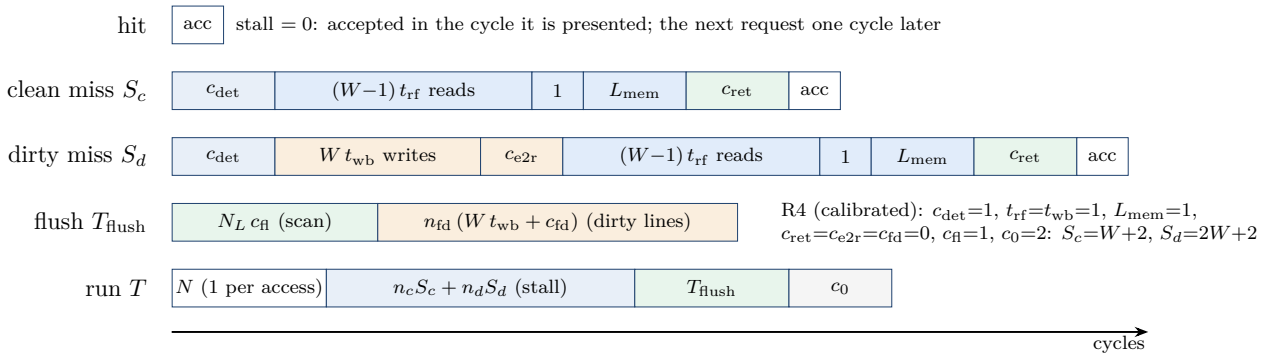


Figure 3: What the timing model adds up for each kind of access (segment lengths schematic). Blue: detection and refill, orange: write-back, green: fixed controller overhead; "acc" is the acceptance cycle, which is not a stall cycle. The functional model decides which row applies; the timing layer only sums the parameters of that row.

4 The Timing Model

4.1 Composition, Not Modification

The T-MOD model is verified, and two tracks depend on its API. Timing is a separate concern: the same functional behavior can have many timings (another controller, another memory), and the DV track does not want timing in its golden reference at all. Therefore the timing model *wraps* the functional model:

- **Composition** (baseline): `cache_timing_model` holds a `cache_model` handle. Every function comes from the wrapped model; the wrapper adds a cycle counter and the timing parameters. Any object with the `cache_model` API — including a future set-associative or prefetching variant — can be wrapped.
- **Inheritance** (alternative): `timed_cache_model extends cache_model` and overrides `access()`, calling `super.access()`. Shorter, but it must also override `clone()` (or a clone loses its timing state), it ties the timing to one functional class, and a T-DV1 scoreboard holding a base handle could silently receive a timed object.

Implement the composition. In the report, show the inheritance version in ten lines and explain which T-MOD review question it relates to.

4.2 API and Implementation

```
class timing_cfg;
  int unsigned c_det = 1, t_wb = 1, c_e2r = 0, t_rf = 1,
               l_mem = 1, c_ret = 1, c_fl = 1, c_fd = 0, c_0 = 0;
  function void load(string fname);      // key=value file from calibration
  function string convert2string();
endclass

class timed_pred #(int ADDR_W = 10, int DW = 32);
  cache_pred #(ADDR_W, DW) pred;         // functional prediction from T-MOD
  int unsigned             stall;        // stall cycles of this access
  longint unsigned         accept;       // cycle at which it is accepted
endclass

class cache_timing_model #(int ADDR_W = 10, int DW = 32);
```

```
  cache_model #(ADDR_W, DW) func;        // T-MOD model, unchanged
  timing_cfg                tcfg;
  longint unsigned          now;         // cycle of the next request
  longint unsigned          stall_cycles;

  function new(cache_model #(ADDR_W, DW) func, timing_cfg tcfg);
    this.func = func; this.tcfg = tcfg; now = 0; stall_cycles = 0;
  endfunction

  function int unsigned miss_stall(result_e r);
    int unsigned W = func.cfg.line_words, s;
    if (r == HIT) return 0;
    s = tcfg.c_det + (W - 1) * tcfg.t_rf + 1 + tcfg.l_mem + tcfg.c_ret;
    if (r == MISS_DIRTY) s += W * tcfg.t_wb + tcfg.c_e2r;
    return s;
  endfunction

  function timed_pred #(ADDR_W, DW) access(cpu_txn #(ADDR_W, DW) t);
    timed_pred #(ADDR_W, DW) p = new();
    p.pred   = func.access(t);           // all function comes from T-MOD
    p.stall  = miss_stall(p.pred.result);
    p.accept = now + p.stall;
    now      = p.accept + 1;             // next request one cycle later
    stall_cycles += p.stall;
    return p;
  endfunction

  // func.flush() writes back; returns T_flush (n_fd from func.stats), advances now
  function longint unsigned flush(ref mem_txn #(ADDR_W, DW) wbs [$]);
  function longint unsigned total_cycles();             // now + c_0 after the flush
  function void clear_stats();                          // end of a warm-up
  function cache_timing_model #(ADDR_W, DW) clone();   // clones func too
endclass
```

4.3 Requirements

T1 `cache_model.svh` and the other T-MOD files are not modified — not even `cache_model_pkg.sv`: the timing classes go into a new package `cache_timing_pkg` that imports `cache_model_pkg` (file list `dv/model/cache_model/cache_timing.f`). The DV track keeps compiling exactly the T-MOD package, and all T-MOD unit tests still pass.

T2 All counts (accesses, misses, write-backs) are read from the wrapped model; the timing layer adds only cycles.

T3 The parameters are run-time values loaded from a calibration file; no latency is a constant in the code.

T4 `access()` returns the stall cycles and the acceptance cycle of every access, so that the model's per-access timeline can be compared with the RTL's (Section 6).

T5 `clone()` is deep and includes the cycle counter; the T-MOD clone check is repeated for the timing model.

T6 A unit test `TEST=timing` in a new top `tb_timing_unit` (`dv/model/cache_model/tb/`; the T-MOD testbench stays unchanged) checks the formulas on hand-computed traces (a hit run, one clean miss, one dirty miss, a flush with two dirty lines) for two parameter sets, with the expected cycles written as literal numbers — re-evaluating the model's own formula in the testbench would check nothing.

T7 With `tcfg.l_mem` and the other parameters at their calibrated values, `tb_explore` in timing mode (`+TIMING=1 +TCFG=<file>`) adds the columns `stall_cycles` and `cycles` to its CSV.

TEST TERM | Unit test

A test of one class (or package) in isolation, without the RTL and without the rest of the testbench, against hand-computed expectations. `TEST=timing` proves that the timing layer implements equations (1)–(4) and that `clone()` is deep *before* the layer is compared with any RTL: a correlation failure can then only come from the parameters or from the RTL, not from arithmetic in the model.

5 Calibration

5.1 Micro-Benchmarks

Calibrate on traces designed to isolate one parameter each, never on the workload traces you will use for validation — otherwise the correlation in Section 6 only proves that a fit can reproduce the data it was fitted to. Add the following patterns to `gen_trace.py`:

TEST TERM | Micro-benchmark (calibration test)

A tiny directed trace built so that one quantity dominates the result: every access of `cal_clean` is a clean miss, every eviction of `cal_dirty` is dirty. It does not check right or wrong by itself; it *measures* one parameter, and its internal consistency (all misses of a class cost the same) is the check. These tests are run on the RTL only for calibration and never reused for validation.

Micro-benchmark	Trace	Measures
`cal_hit`	One miss, then 1000 reads of the same line	Issue rate of hits (must be 1 per cycle)
`cal_clean`	Reads cycling through $k > 1$ tags on one index (all clean)	$S_c(W)$
`cal_dirty`	Writes cycling through $k > 1$ tags on one index (every eviction dirty)	$S_d(W)$
`cal_flush`	Fill j dirty lines ($j = 0, 1, N_L/2, N_L$), then end of trace	c_{fl}, c_{fd}
`cal_mix`	Alternating clean and dirty misses	Interaction: does a miss directly after a miss cost the same?

Run each on `tb_replay` for `line_words` $\in \{2, 4, 8, 16\}$ and read the stall cycles of every access from the per-access log that T-AM1 added to `tb_replay`. Within one micro-benchmark, all misses must have the same stall count; if they do not, find out why before fitting anything.

5.2 Fitting

For each penalty, fit $S(W) = a + bW$ over the four line sizes by least squares:

$$b = \frac{\sum_i (W_i - \bar{W})(S_i - \bar{S})}{\sum_i (W_i - \bar{W})^2}, \qquad a = \bar{S} - b\,\bar{W}.$$

For a correct blocking controller the residuals of this fit are zero: the stall counts are integers determined by the FSM. Then map the fitted coefficients back to the parameters: by equations (1) and (2), $b_c = t_{\mathrm{rf}}$, $b_d - b_c = t_{\mathrm{wb}}$, $a_c = c_{\mathrm{det}} - t_{\mathrm{rf}} + 1 + L_{\mathrm{mem}} + c_{\mathrm{ret}}$ and $a_d - a_c = c_{\mathrm{e2r}}$. Only the sums $c_{\mathrm{det}} + c_{\mathrm{ret}}$ are identifiable from stall counts; split them using the waveform of one miss — or let `tb_replay` log, per access, the cycles of the first memory request, the last refill read and the last read response (the reference solution does this; it measures $c_{\mathrm{det}} = 1$, $L_{\mathrm{mem}} = 1$, $c_{\mathrm{ret}} = 0$ for the R4 controller).

SYNOPSYS CLOUD

The waveform of one miss: `make -C dv/model/explore sim TB_TOP=tb_replay TRACE=<cal trace> PARAMS="LINE_WORDS=4 NUM_LINES=16" WAVES=1`, then `make waves` with the same `TB_TOP` and `PARAMS` opens Verdi on the FSDB; put `cpu.req_valid`, `cpu.req_ready`, `state_q` and the `mem` handshake side by side. All calibration and correlation runs use VCS (`SIM=vcs`); the timing model adds no measurable cost to the functional model (the full what-if sweep of Section 7, 6 traces × 4 configurations × 6 latencies × 6 model variants, runs in about 35 s).

Quantity	W=2	W=4	W=8	W=16	**Fit**	**FSM explanation**
S_c (cycles)					$a_c + b_c W$	
S_d (cycles)					$a_d + b_d W$	
T_{flush}, 0 dirty					$N_L c_{\mathrm{fl}}$	
T_{flush}, all dirty						

Explain every parameter by the FSM. A calibrated parameter that you cannot explain from the R4 controller ("$c_{\mathrm{ret}} = 2$ because the REFILL state waits one cycle for the last response and IDLE needs one cycle to see the new tag") is a fitted number, not a model; it will be wrong for the next configuration. Write the explanation in the column of the table, referring to the state diagram in `hw/ip/dcache/doc/design.md`.

Cross-check with V3. Lab V3 measured minimum, average and maximum acceptance latency per test. Predict these numbers with the calibrated model for the V3 `sequential` and `conflict` patterns (generate equivalent traces) and compare. The maximum latency must equal $1 + S_d(W)$; the bounded-latency assertion of V3 must be consistent with it.

6 Cycle Correlation

6.1 Error Metrics

Run all six T-AM1 workload traces on `tb_replay` and on the timing model, for the default configuration and two other configurations (at least one with a different `line_words`). For every trace report the relative error of the total cycles and of the stall cycles:

$$e_T = \frac{T_{\mathrm{model}} - T_{\mathrm{RTL}}}{T_{\mathrm{RTL}}} \times 100\%, \qquad e_S = \frac{S_{\mathrm{model}} - S_{\mathrm{RTL}}}{S_{\mathrm{RTL}}} \times 100\%.$$

The target is $|e_T| < 2\%$ on every trace for this blocking cache. Look at e_S as well: on a trace with a 95% hit rate, hits dominate T, so a 20% error in stall cycles shows up as only a few percent in T. e_T is what the architect sees; e_S is what tells you whether the model of misses is right. Explain every $|e_S| > 5\%$. Also report the mean absolute error over traces and the worst trace; a model can have a small average error and one large outlier that matters.

Trace	**Config**	T_{RTL}	T_{model}	e_T	S_{RTL}	S_{model}	e_S
`seq`	16×4						
`random`	16×4						
`conflict`	16×4						
`loop`	16×4						
`fits`	16×4						
`mixed`	16×4						
...							

6.2 Residual Analysis Access by Access

An end-of-trace error says *that* the model is off, not *where*. Both `tb_replay` and the timing model write a per-access log (access number, result, stall cycles, acceptance cycle). `dv/model/explore/cycle_diff.py` — named after its CVA6 counterpart — aligns the two logs by access number and computes, for each access, the difference of the *inter-acceptance intervals*:

$$\delta_i = (c_i^{\text{model}} - c_{i-1}^{\text{model}}) - (c_i^{\text{RTL}} - c_{i-1}^{\text{RTL}}).$$

Comparing intervals rather than absolute cycles prevents one early error from showing up as a difference at every later access. The script prints the first access with $\delta_i \neq 0$, and a histogram of δ_i grouped by (result of access $i-1$, result of access i): residuals that depend on the *previous* access are the typical signature of a missing interaction term.

```
# dv/model/explore/cycle_diff.py (excerpt)
def deltas(model_log, rtl_log):
    m, r = read_log(model_log), read_log(rtl_log)   # lists of (idx, result, accept)
    assert [x.result for x in m] == [x.result for x in r], "counts differ: fix T-AM1
    first"
    for i in range(1, len(m)):
        d = (m[i].accept - m[i-1].accept) - (r[i].accept - r[i-1].accept)
        yield i, (m[i-1].result, m[i].result), d

groups = collections.defaultdict(collections.Counter)
for i, key, d in deltas(sys.argv[1], sys.argv[2]):
    groups[key][d] += 1
```

Typical sources of residuals in a blocking cache, and what to do with them:

- **A one-cycle bubble after a miss**: the first hit after a refill costs one more cycle than other hits (the read response of the held request, or IDLE re-evaluating the tag). Shows up in the group (miss, hit). Add a parameter; explain it by the FSM.
- **Miss directly after a miss**: the controller may skip or add a cycle when it goes from REFILL back to EVICT. Group (miss, miss).
- **Write versus read requests**: a write hit and a read hit may not have the same issue rate if the read response occupies a resource. Group by operation as well as result.
- **Start and end effects**: reset release, the first request, the flush handshake. These are constant (c_0) and vanish relative to long traces; do not hide a real per-access error in c_0.
- **Measurement errors**: the RTL log samples the cycle at a different point than the model defines (acceptance edge versus the cycle after). Fix the definition, not the model.

Every residual must end in one of three states, recorded in the report: *modeled* (a parameter was

added and calibrated on a micro-benchmark), *accepted* (explained and below the target), or *bug* (in the RTL, the model or the testbench, fixed with a commit referencing the trace).

7 What-If Studies

With a calibrated and correlated model you can now answer questions that would need RTL changes. State for every result that it is a *prediction outside the correlated range*: the model was validated for $L_{\text{mem}} = 1$ and a blocking cache only.

7.1 Memory Latency

`arb_mem` answers in one cycle; a real memory system (an L2, a DRAM controller) takes ten to a few hundred cycles. Sweep $L_{\text{mem}} \in \{1, 2, 5, 10, 20, 50\}$ with all other parameters calibrated, on all six traces and the three best configurations of your T-AM1 memo. From equations (1) and (3), AMAT grows linearly in L_{mem} with slope equal to the miss rate:

$$\frac{\partial\, AMAT}{\partial L_{\text{mem}}} = m_c + m_d.$$

Plot cycles per access versus L_{mem}. Answer: does the T-AM1 recommendation (chosen on hit rate and area) stay the best configuration when $L_{\text{mem}} = 20$? At which latency do two configurations cross, and why do longer lines become relatively more attractive as latency grows (the latency is paid once per line, the transfer once per word)?

7.2 A Non-Blocking Cache with MSHRs

A non-blocking cache keeps accepting requests while misses are outstanding. Each outstanding miss occupies a **miss status holding register** (MSHR); with N_{MSHR} registers, up to N_{MSHR} misses overlap. A request to a line that is already being refilled (a *secondary miss*) merges into its MSHR. There is no RTL for this; you approximate it in the model, in two ways, and compare them.

(a) Closed-form bound. Let S_i be the stall of miss i in the blocking model and $B = \sum_{\text{misses}} (W t_{\text{rf}} + [\text{dirty}]\, W t_{\text{wb}})$ the number of cycles the single memory port is busy. With N_{MSHR} registers, at most $N_{\text{eff}} = \min(N_{\text{MSHR}}, MLP)$ misses overlap, where MLP (memory-level parallelism) is the average number of misses that occur within $\bar{S}$ accesses after a miss — a property of the trace that the functional model can compute. Then

$$T_{N_{\text{MSHR}}} \;\approx\; \max\Big(N + \frac{\sum_i S_i}{N_{\text{eff}}},\; N + B\Big)$$

— latency is divided among overlapping misses, but the memory port cannot transfer more than one word per cycle.

(b) Event approximation. Extend `cache_timing_model` by a derived class `mshr_timing_model` (it may override `access()`; the base timing class stays unchanged for the blocking correlation):

```
// per access, at cycle 'now'
if (pred.result == HIT && !in_flight(line)) now += 1;  // hit under miss
else if (in_flight(line))                   now += 1;  // secondary: merge
else begin                                             // primary miss
  if (busy_mshrs() == n_mshr)                          // structural stall
    now = max(now, earliest_free());
  start     = max(now, port_free);                     // single memory port
  port_free = start + occupancy(pred);                 // W*t_rf (+ W*t_wb)
  allocate_mshr(line, start + miss_stall(pred.result)); // free when refilled
```

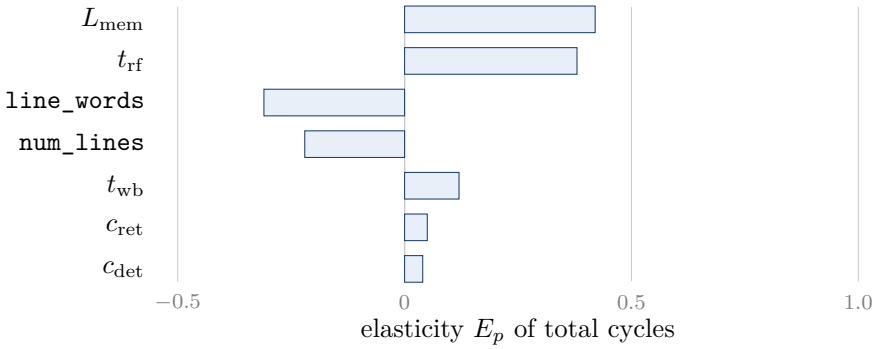


Figure 4: Form of the required tornado chart: parameters sorted by the magnitude of their elasticity for one trace. The values shown are illustrative only; yours come from the model.

```
  now += 1;
end
```

Assumptions, which you must state with the results: accesses are independent (no instruction waits for load data — this makes the result an *upper bound* on the benefit); the functional result of each access is taken from the blocking model (for this cache the miss/hit sequence does not depend on timing, since requests are processed in order and lines are allocated at the miss); write-backs use the same port and are not buffered.

Consistency check: with $N_{\text{MSHR}} = 1$ and hit-under-miss disabled (`+HIT_UNDER_MISS=0`, or, as in the reference `tb_explore`, an extra model variant run automatically next to every blocking run), `mshr_timing_model` must reproduce the blocking model's cycle count exactly.

TEST TERM | Consistency check

A comparison of two models that must agree in a limiting case: the non-blocking model with one MSHR and no hit-under-miss *is* the blocking cache. There is no RTL for the non-blocking cache, so this is the only test that connects it to anything correlated; a failure means the event model is wrong even where its answer is known.

Then plot cycles per access versus $N_{\text{MSHR}} \in \{1, 2, 3, 4\}$ for $L_{\text{mem}} \in \{1, 10, 50\}$ on every trace, with (a) and (b) on the same plot. Answer: on which traces do extra MSHRs help, and why not on the others (MLP, bandwidth bound)? Where do (a) and (b) disagree, and which one do you trust more?

8 Sensitivity Analysis

A model has many parameters; not all of them matter. The **elasticity** of total cycles T with respect to a parameter p is the relative change of T per relative change of p, estimated with a central difference ($\delta = 10\%$, rounding integer parameters to the nearest value that changes):

$$E_p = \frac{\partial T/T}{\partial p/p} \approx \frac{T\big(p(1+\delta)\big) - T\big(p(1-\delta)\big)}{2\,\delta\,T(p)}.$$

$E_p = 0.3$ means a 10% error in p changes the predicted cycles by 3%. Compute E_p for every timing parameter and for `line_words` and `num_lines` on every trace, and draw one tornado chart per trace (Figure 4 shows the form).

Answer in the report:

1. Which parameters must be calibrated precisely, and which could be wrong by 50% without changing any conclusion? Does the answer depend on the trace?

2. Is the *ranking* of your three best configurations from the T-AM1 memo stable when each parameter is changed by ±20%? A recommendation that flips under a 20% parameter error is not a recommendation.
3. Why is the elasticity of L_{mem} close to $\frac{(m_c+m_d)\,L_{\text{mem}}}{AMAT}$? Derive it from equation (4).

9 Limits of Cycle-Approximate Models

The model of this lab reaches a fraction of a percent on a blocking cache with a fixed-latency memory, because every interaction is captured by a few parameters. Real systems add effects that a cycle-approximate model represents only statistically or not at all:

Effect	**Why it is hard to approximate**	**What models do**
Contention (a second master on `arb_mem`, a shared bus)	Delay depends on the exact cycle alignment of two streams	Average queueing delay; or a cycle-accurate arbiter
Variable memory latency (DRAM row hits/misses, refresh)	Latency depends on address history and time	Simple DRAM model (open-row table) or a DRAM simulator
Core pipeline dependencies (load-use, branches)	Whether a miss stalls depends on the instructions after it	Trace annotated with dependencies; a core model
Out-of-order completion, MSHR merging	Order of completion changes which later accesses stall	Event-driven models (Section 7 b)
Micro-architectural corner cases (bubbles, replays)	Each is small and specific to one RTL version	Add a parameter only if correlation shows it

Three rules follow from this lab:

- **Correlation is per workload and per configuration.** A model correlated on synthetic traces is not validated on real software; a model correlated at $L_{\text{mem}} = 1$ is an extrapolation at $L_{\text{mem}} = 50$. State the range.
- **Correlation decays.** Every RTL change can shift a parameter. Re-run the calibration and correlation in the CI-style check (`make check-all`), as T-AM1 did for the counts.
- **Errors compose.** The error of a what-if result is not the correlation error: it includes the error of the assumptions that were never correlated (independence of accesses, port model). Present what-if results as ranges or rankings, not as three-digit numbers.

A cycle-accurate model avoids the approximations but must reproduce every bubble of the RTL; it is worth building when the performance question is about those bubbles (pipeline tuning) and not about architecture. Many teams keep both: an approximate model for breadth and a cycle-accurate one (or the RTL itself, on FPGA) for depth.

10 Industrial Reference: the CVA6 Performance Model

The CVA6 repository (OpenHW Group) contains, in `perf-model/`, a performance model of the core written in Python (its README calls it a cycle-accurate model of the CVA6 control path), and a script, `cycle_diff.py`, used to compare the model's timing with traces produced by RTL simulation. The model is driven by the instruction trace of an RTL run and predicts, instruction by instruction, when each instruction is issued and committed; `cycle_diff.py` turns a trace (the RTL's, or the model's annotated one) into per-instruction durations, and the two outputs are compared side by side to see where the predicted and the measured cycle numbers diverge. (Directory content as at the time of writing; read the directory's README for the current state.)

The structure is the one you built, one level up:

This lab	**CVA6 `perf-model/`**
T-MOD functional model + timing layer	Python model of the core's timing, driven by an instruction trace
Address trace (`dv/model/traces/`)	Instruction trace from an RTL simulation
`tb_replay` per-access log	RTL trace log with cycle numbers
`cycle_diff.py`: per-access interval differences	`cycle_diff.py`: per-instruction cycle differences
Calibrated parameters (`timing_cfg`)	Model parameters (latencies, issue rules)

Read the CVA6 `perf-model/` directory (clone the repository; no simulation is required) and write half a page in the report: what the model predicts, what it takes from the RTL trace and what it predicts on its own, how its comparison script differs from yours, and one idea you would adopt.

11 Requirements and Tasks

1. Derive equations (1)–(4) for *your* R4 controller from its state diagram, before any measurement, and write down your predicted parameter values.
2. Implement `timing_cfg` and `cache_timing_model` by composition (T1–T7) and the unit test `TEST=timing`.
3. Add the calibration micro-benchmarks to `gen_trace.py`; run them on `tb_replay` for four line sizes; fit and explain every parameter by the FSM; compare with your predictions from task 1 and with the V3 latency statistics.
4. Correlate cycles on all six workload traces for three configurations; reach $|e_T| < 2\%$ per trace; report e_S; write `cycle_diff.py` and classify every residual as modeled, accepted or bug.
5. Run the memory-latency sweep and answer its questions.
6. Implement the closed-form MSHR bound and `mshr_timing_model`; pass the $N_{\mathrm{MSHR}} = 1$ consistency check; produce the MSHR plots and answer the questions.
7. Compute the elasticities, draw the tornado charts, and test the stability of the T-AM1 ranking.
8. Read the CVA6 `perf-model/` directory and write the comparison.
9. Write the limits section of the report: correlated range, uncorrelated assumptions, and what you would need to validate the what-if results.

12 Repository and Flow

```
dv/model/cache_model/              (T-MOD files unchanged)
+-- cache_timing_pkg.sv        new package: import cache_model_pkg::*; `include of:
+-- timing_cfg.svh             new
+-- timed_pred.svh             new (prediction + issue/stall/accept cycles)
+-- cache_timing_model.svh     new (composition around cache_model)
+-- mshr_timing_model.svh      new (derived from cache_timing_model, what-if only)
+-- cache_timing.f             new: -f cache_model.f + cache_timing_pkg.sv
+-- tb/tb_timing_unit.sv       new: TEST=timing (tb/tb_timing.f)
dv/model/traces/
+-- gen_trace.py               + cal_hit, cal_clean, cal_dirty, cal_flush, cal_mix
dv/model/explore/
+-- tb_explore.sv              + timing mode: +TIMING=1 +TCFG= +MEM_LAT=,
|                                what-if: +WHATIF=1 +MEM_LATS=1,2,5 +MSHRS=1,2,4, +SENS=<
    pct>
```

```
+-- tb_replay.sv              per-access log (from T-AM1), unchanged
+-- calibrate.py              least-squares fit -> timing_params.txt
+-- timing_params.txt         calibrated parameters (key=value), with RTL commit id
+-- cycle_diff.py             per-access residuals, grouped histogram
+-- whatif.py  sensitivity.py plots for latency sweep, MSHR, tornado charts
+-- results/                  calibration, correlation, what-if CSV and PNG
+-- doc/cycle_model.md        the lab report
```

The Makefile of `dv/model/explore/` from T-AM1 gains five targets; the shared flow targets and variables are unchanged:

Target	Function
`calibrate`	`replay` of the micro-benchmarks for every `LINE_WORDS` in `CAL_LW`; `calibrate.py` writes `timing_params.txt`
`cycles`	`replay` and timing-model runs of all workload traces × `replay_configs.txt`; error table; exit code $\neq 0$ if any $\lvert e_T \rvert$ exceeds `TOL` (default 2)
`whatif`	Timing-model sweeps of `MEM_LAT` and `MSHR`; plots
`sensitivity`	Elasticities and tornado charts
`timing-unit`	`sim TB_TOP=tb_timing_unit TEST=timing` (the unit test of T6)

```
source env.sh
make -C dv/model/explore timing-unit
make -C dv/model/explore calibrate CAL_LW="2 4 8 16"
make -C dv/model/explore cycles TOL=2
D=build/explore/LINE_WORDS-4_NUM_LINES-16/sim/tb_replay
$PYTHON dv/model/explore/cycle_diff.py $D/mixed_model.acc.csv $D/mixed_rtl.acc.csv
make -C dv/model/explore whatif MEM_LAT="1 2 5 10 20 50" MSHR="1 2 3 4"
make -C dv/model/explore sensitivity
```

Run `make -C dv/model/explore cycles TRACES=mixed` as a CI step next to `check-all` (which itself runs `lint` and `sim` of every `dv/model/*/Makefile`, i.e. the count replay of T-AM1). Git: branch `lab/tam2-cycle-model`, tag `tam2-done`. `timing_params.txt` records the RTL commit (and the SHA-256 of `dcache.sv`) it was calibrated on; `make cycles` warns when the RTL has changed since.

TEST TERM | Regression (CI correlation check)

Re-running the calibration and the cycle correlation automatically after every change. Correlation decays: an RTL change that adds one cycle to a refill silently invalidates every what-if study; the CI step turns it into a failing check the day it happens.

SYNOPSYS CLOUD

Everything in this lab runs on VCS (`SIM=vcs`): the class-based model, the timing package and the RTL replay; Verilator 5.013 cannot compile the T-MOD package. Lint the timing package with `make -C dv/model/explore lint` (VCS `+lint=all`). The Python scripts run with `$(PYTHON)` (`python3.11` on the cloud); without `matplotlib` they write the plots as SVG/PNG themselves.

13 Deliverables

The merge request contains the new classes, the unit test, the scripts, the calibrated parameter file, the results and `dv/model/explore/doc/cycle_model.md` with:

- derivation of the cycle formulas from the R4 FSM, predicted parameter values, and the calibration table with an FSM explanation of every parameter;

- class diagram (`cache_model`, `cache_timing_model`, `mshr_timing_model`) and the inheritance alternative;
- cycle correlation table (e_T, e_S, mean and worst), residual classification, `cycle_diff.py` output for the worst trace;
- latency and MSHR plots with answers, assumptions and the consistency check; tornado charts and ranking stability;
- the CVA6 comparison, the limits section, and an update of the T-AM1 memo if cycles change the recommendation.

14 Assessment

Criterion	Weight
Timing formulas derived from the FSM; timing model by composition, unit test	20%
Calibration: micro-benchmarks, fit, every parameter explained	20%
Cycle correlation: error per trace within target, residual analysis	25%
What-if studies: memory latency, MSHR approximation and consistency check	15%
Sensitivity analysis and ranking stability	10%
Limits discussion, CVA6 comparison, report quality	10%
Total	100%

15 Review Questions

1. Why is the timing added by composition around the T-MOD model rather than by editing it? What would the DV track lose otherwise?
2. Why are the clean and dirty miss penalties linear in `line_words`? Which FSM property would break that?
3. Why must calibration traces be different from validation traces? What does a perfect correlation on the calibration traces prove?
4. Only $c_{\text{det}} + c_{\text{ret}}$ is identifiable from stall counts. Why, and does it matter for the predictions?
5. A trace has $e_T = 0.4\%$ and $e_S = 12\%$. Is the model good enough? What do you look at next?
6. Why does `cycle_diff.py` compare intervals between acceptances instead of absolute acceptance cycles?
7. What does the slope of AMAT versus L_{mem} tell you, and why do longer lines become more attractive at high latency?
8. Why is the MSHR event approximation an upper bound on the benefit of a non-blocking cache? Which property of real programs does it ignore?
9. A parameter has elasticity 0.02 on every trace. How precisely must it be calibrated?
10. Name two effects a cycle-approximate model of a real memory system cannot capture with a fixed latency, and how a performance team would handle each.

16 References

- Hennessy and Patterson, *Computer Architecture: A Quantitative Approach*: Chapter 2 and Appendix B (miss penalty, non-blocking caches, MSHRs, memory-level parallelism).
- D. Kroft, "Lockup-Free Instruction Fetch/Prefetch Cache Organization", *ISCA*, 1981 (the original MSHR paper).
- L. Eeckhout, *Computer Architecture Performance Evaluation Methods*, Morgan & Claypool, 2010 (model types, sampling, validation).
- B. Black and J. P. Shen, "Calibration of Microprocessor Performance Models", *IEEE Computer*, 1998.
- CVA6 repository (OpenHW Group), directory `perf-model/`: Python performance model and `cycle_diff.py`.
- The gem5 simulator documentation: classic and Ruby memory systems, as an example of models at several levels of detail.
- R. Jain, *The Art of Computer Systems Performance Analysis*, Wiley, 1991 (sensitivity analysis, presenting results).